

Resume



안녕하세요 이영민입니다.

Email

bbok4yo@gmail.com

Channel

[Github](#)

[Velog](#)

ABOUT ME

- React/Next.js (App Router) 기반 프로젝트에서 동적 라우팅 및 전역 상태 관리를 포함한 아키텍처를 설계하고 구축합니다.
- Git/PR 기반 협업에 익숙하며, 기획 의도를 명확히 파악하는 세밀한 커뮤니케이션을 주도합니다.
- 컴포넌트 재사용성과 API 호출 타이밍 등 실제 서비스 관점에서 구조 설계를 주도하며 성능 최적화를 추진합니다.
- 함께 일하고 싶은 동료로서, 기술과 더불어 좋은 팀 문화를 만드는 데 적극 기여합니다.
- 백엔드 로직과 서버 환경에 대한 이해를 바탕으로 데이터 구조를 함께 설계하며, 효율적인 API 연동을 주도합니다.
- TypeScript 도입 및 성능 최적화를 통해 코드의 안정성을 확보하고, 사용자에게 쾌적한 디지털 경험을 제공합니다.
- AI와 협업하여 요구사항 분석부터 SDD 설계, 자동화된 코드 품질 검증까지 소프트웨어 엔지니어링 전 과정을 최적화합니다.

CAREERS

리얼라인

기간: 2026.01 ~ 2026.02

- AI 기반 소프트웨어 엔지니어링 파이프라인 구축: 단순 코드 생성을 넘어 요구사항 분석, SDD 설계, 구현·품질 검증 전 과정에 Claude Code를 통합하여 개발 주기를 단축했습니다.
- 단기 다중 프로젝트 완수: 1개월 내 전략 보고서 생성기·지도 데이터 시각화·스팟업 웹 3종을 스펙 기반 AI 워크플로우로 구현했습니다.

성과: AI를 활용한 일관된 개발 및 품질 엔지니어링

1. AI-Assisted SDD 개발 및 문서화

- 아키텍처 설계: 요구사항을 바탕으로 Claude Code와 상호작용하며 시스템 아키텍처, 데이터 모델링, 컴포넌트 계층을 정의하는 SDD를 신속하게 작성했습니다.
- 문서화 자동화: API 명세 변경, 상태 관리 플로우 등을 AI를 통해 실시간으로 반영하여 프로젝트 가시성과 협업 효율성을 확보했습니다.

2. Custom Skill-Hook 기반 코드 품질 엔지니어링

- **자동화 품질 검증:** Custom Skill과 Git Hook(Pre-commit/Pre-push)을 연동하여 커밋 단계에서 AI가 정적 분석·컨벤션 검사를 자동 수행하도록 구성했습니다.
- **지속적 리팩토링:** AI가 프로젝트 전체 컨텍스트를 파악하여 보안 취약점 점검, 중복 코드 제거, 아키텍처 개선안을 선제적으로 제안하고 적용했습니다.

라운피플

기간: 2024.01 ~ 2024.04

성과: 테스트 케이스(TC) 및 주어진 테스트 항목에 기반한 기능 테스트 수행

- JIRA를 활용해 버그 리포트를 생성하고 이슈를 트래킹했습니다.
- 테스트 결과 보고서를 작성하고 개발팀에 피드백을 전달했습니다.
- 관련 기술: JIRA

ACTIVITIES

프론트엔드 개발자 과정

소속/기관: Kakao x goorm

활동 연도: 2024.07~2025.04

- Next.js를 학습하고 실제 프로젝트에 적용해 실전 감각을 키웠습니다.
- 디자이너, PM, 백엔드 개발자 등 다양한 직군과 협업하면서 직군별 의사결정 우선순위 차이를 좁히는 커뮤니케이션 방식을 익혔습니다.

캠퍼스 SW 아카데미 TABA 빅데이터 분석 및 인공지능 처리를 위한 SW융합개발자양성

소속/기관: 과학기술정보통신부

활동 연도: 2023

- 프론트엔드·백엔드 개발 및 Figma 기반의 기획 등 전반적인 개발 협업 과정을 경험했습니다.
- 협업 톨과 개발 전반을 아우르며 실제 프로젝트에서의 기본적인 협업 프로세스를 이해했습니다.
- 과정 수료 후 프로젝트 부분에서 우수상 및 우수교육생으로 선정되었습니다.

EDUCATION

- 단국대학교 컴퓨터공학과 : 2019.03. ~ (졸업예정)

STACKS

Front-End

- HTML, CSS(SCSS), JS(ES6) & TS
- React.js, Next.js
- tailwind
- Redux Toolkit, react-query, zustand

- D3.js

Collaboration & Tools

- slack
- Figma
- Git, Github

PROJECTS

MindGraph (AI 지식 캡처 & 그래프 시각화)

서비스 개요: ChatGPT·Gemini·Claude·Grok 4개 LLM 답변을 hostname 자동 감지로 캡처해 의미 단위로 묶고 지식 그래프로 시각화하는 Chrome Extension·Next.js 웹앱입니다. 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

팀원: 1명 (1인 AI 협업 운영)

기술 스택: TypeScript, Next.js 16, React 19, D3.js, Claude Code, Chrome Extension Manifest V3, IndexedDB, Supabase, GitHub Actions, LLM API (OpenAI·Anthropic·Google·xAI)

- 4 LLM의 잦은 UI 변경에 단일 파일 수정으로 대응하기 위해 답변 DOM 선택자를 detector.ts SELECTORS에 모으고 hostname 자동 감지로 통합 레이어를 만들었습니다.
- Claude Code 위에 9개 혹(차단 4·보조 5)과 plan·qa·ship 3 phase /sprint·4중 SSOT 자동 동기를 설계해 sprint 라이프 사이클 반복 작업을 한 명령 흐름으로 자동화했습니다.
- 매 prompt에 전체 docs가 끌려오는 컨텍스트 과부하를 줄이기 위해 7부서·9에이전트별 docs를 분리하고 task_type·코드 path glob을 must-read doc에 매핑해 첨부량을 5개 내외로 줄였습니다.
- 추천 정확도와 클릭률을 출시 전에 측정하기 위해 lib/telemetry.ts에 AccuracyEvent(top-3 ≥ 60%)·InteractionEvent(클릭률 ≥ 60%)와 nodeId 한정 적재 정책을 함께 설계했습니다.
- 네트워크 단절 시 LLM 답변 캡처 유실을 막기 위해 IndexedDB 우선 기록과 Pending Ops 큐(MAX_RETRIES=5) 기반 Write-Behind 동기화로 오프라인 CRUD와 온라인 복귀 자동 flush를 구현했습니다.
- 노드 정착이 느린 문제를 해결하기 위해 이진 탐색·코사인 법칙 기반 궤도 계산과 alphaDecay를 0.012에서 0.028로 조정해 정착 tick을 약 340에서 140으로 단축했습니다.

chatGraph (AI 대화 시각화 플랫폼)

서비스 개요: 선형 채팅 UI에서 사고의 분기와 맥락이 사라지는 한계를 해결하기 위해, LLM과의 대화를 꼬리물기 형식의 마인드 맵 그래프로 시각화하여 비선형 사고 흐름을 보존하고 대화의 깊이와 관계를 한눈에 보여주는 웹 서비스

팀원: 4명 (FE 2명, BE 2명)

기술 스택: TypeScript, React, Next.js, D3.js, Zustand, Tailwind CSS, TanStack Query, Vitest

- LLM 응답 시점이 매번 달라지는 환경에서 입력 직후 화면이 멈추지 않도록 sessionStorage에 프롬프트를 저장하고 temp ID로 즉시 라우팅한 뒤 mutation 성공 시점에 실제 ID로 교체하는 Optimistic 라우팅 흐름을 구현했습니다.
- 4인 협업에서 새 토픽 시작·토픽 전환·노드 경로 탐색이 회귀로 무너지지 않도록 findPathToNode와 Zustand 토픽 토클, 새 토픽 시작 후에 Vitest 회귀 테스트를 작성해 리팩토링 중에도 LLM 통신 경로 정확함을 자동 검증하도록 했습니다.
- 단일 파일에 843줄까지 비대해진 혹을 책임 단위로 쪼개 355줄까지 축소하고, Feature-First 디렉토리 구조로 도메인 (features)·조립(views)·공통(shared)을 분리하여 LLM 보조 리팩토링이 안전하게 진입할 수 있는 모듈 경계를 마련했습니다.

다.

- 거대해진 페이지 컴포넌트의 책임을 TopicContentLayout·StandardChatView·OptimisticChatView로 뷰 계층을 분리하고 useSuspenseQuery 사용 분기에 따라 Suspense 경계와 컴포지션을 재설계해 LLM 응답 상태별 뷰 분기를 명시적인 컴포넌트 경계로 정리했습니다.
- 사이드바에서 토픽 클릭 시 발생하던 초기 로딩 지연을 줄이기 위해 Zustand 스토어에 응답을 프리패치해 두고 useSuspenseQuery의 initialData로 주입하는 패턴을 적용해 라우트 전환 시 빈 화면 없이 데이터가 즉시 그려지도록 구현했습니다.
- 팀원이 도입한 D3.js Force Simulation이 React 리렌더 시 SVG 트리와 충돌하는 문제를 해결하기 위해 useEffect cleanup과 `d3.selectAll("*").remove()` 패턴을 결합해 데이터 변경 시 SVG를 다시 그리는 라이프사이클 통합 코드를 담당했습니다.